PROJECT REPORT ON

CITY NAVIGATION SYSTEM

By

A24126510267- B.SUBHASHINI

A24126510268- G.HASINI

A24126510269- P.HARSHAVARDHAN

A24126510270- S.SRAVANI

SUBMITTED TO

Mr.B MAHESH M.Tech,(Ph.D)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY &SCIENCES (A)

(*Affiliated to Andhra University)*

Sangivalasa 531162, Bheemunipatnam mandal, Visakhapatnam Dist.

CERTIFICATE:

This is to certify that the project titled “city navigation system-Finding shortest path using Dijkstra’s Algorithm” has been carried out by

A24126510267-B.SUBHASHINI

A24126510268-G.HASINI

A24126510269- P.HARSHAVARDHAN

A24126510270- S.SRAVANI,

Students of CSE-C , under the guidance of MAHESH SIR during the academic year 2025–2026.

This work is original and has not been submitted for any other degree or diploma.

Faculty Guide: _____

(Signature & Seal)

Head of Department: _____

(Signature & Seal)

ACKNOWLEDGEMENT:

We would like to express our sincere gratitude to Mahesh sir, our project guide, for their valuable guidance, support, and encouragement throughout this project.

We also thank the Department of Computer Science and Engineering, ANITS engineering college , for providing the necessary facilities and resources to complete this project successfully.

Lastly, we extend our gratitude to all faculty members, friends, and family who contributed to the success of this work.

ABSTRACT :

This project, titled “City Navigation System— Finding Shortest Path using Dijkstra’s Algorithm,” aims to find the shortest path between two nodes (or cities) in a weighted graph. The core computation is performed in C language using Dijkstra’s algorithm, which ensures efficient and accurate shortest path determination for non-negative edge weights.

In today’s world, navigation systems have become a crucial part of daily life. The 'City Navigation System' aims to help users find the shortest path between two cities based on distance or travel time. This system uses the concepts of graphs, where cities are represented as vertices and routes between them as edges.

By implementing Dijkstra’s algorithm, the system efficiently determines the optimal route between two destinations. Additionally, weighted graphs can be extended with live traffic-like parameters to simulate real-time conditions. This project demonstrates the practical application of graph algorithms and data structures in real-world scenarios.

TABLE OF CONTENTS:

S.no	Sections	Page no:
1.	Introduction	6-7
2.	Problem Definition & Objectives	8
3.	System Design/Methodology	9-11
4.	Implementation Details	12-23
5.	Results and Discussion	24-26
6.	Conclusion	27
7.	References	28
8.	Appendix	29-36

1]INTRODUCTION:

Transportation and navigation play an essential role in the modern world. With rapid urbanization and technological advancements, people increasingly rely on navigation systems to determine the most efficient routes. The City Navigation System is designed to simulate a simplified model of such systems using the principles of graph theory and algorithms.

Graphs serve as a natural representation of cities and roads, making them ideal for modeling complex networks. In this project, we represent cities as nodes and roads connecting them as weighted edges. The weights correspond to parameters such as distance or time, enabling the system to compute the most efficient path.

Dijkstra's algorithm is employed for computing the shortest path. It systematically explores all possible routes and determines the path with the minimum cumulative weight. The implementation showcases the importance of algorithmic efficiency in navigation and logistics systems.

Motivation:

The rapid growth of urbanization has led to increased traffic congestion and longer travel times.
People often face challenges in finding the most efficient routes to reach their destinations.
With the widespread use of GPS-based systems and online maps, it has become essential to understand the underlying algorithms that power these navigation tools.

This project, *City Navigation System*, is motivated by the need to explore how data structures and algorithms — especially graph-based techniques — can be applied to solve real-world routing problems.

By building our own version of a navigation system using Dijkstra's Algorithm, we gain hands-on experience with one of the most fundamental concepts in computer science: **shortest path computation**.

Project Overview:

The **City Navigation System** is a console-based program that simulates the working of a navigation tool.
It allows users to input a set of cities and the distances or travel times between them.
Using **graphs** as the underlying data structure, cities are represented as vertices and the roads connecting them as weighted edges.
The **Dijkstra's Algorithm** is implemented to compute the shortest path between a selected source and destination city.

The project consists of multiple modules such as:

1. **Data Input Module** – Accepts the number of cities, names, and interconnections.
2. **Graph Construction Module** – Builds an adjacency matrix or adjacency list representation.
3. **Algorithm Module** – Applies Dijkstra's Algorithm to compute minimum distance.
4. **User Interaction Module** – Allows user to select source and destination dynamically.
5. **Output Module** – Displays the shortest route and total distance/time.

The system is designed for **flexibility and extendibility**.
Future enhancements can include adding live traffic updates, a GUI interface, or integration with real-world APIs such as Google Maps.
Through this project, we aim to bridge the gap between theoretical algorithmic knowledge and real-world application development.

2]Problem statement:

Finding the shortest path between two cities is an important problem in logistics and navigation. Manual route selection can be inefficient; thus, an automated system using graph algorithms provides a more reliable solution.

1.Existing System

Traditional methods or maps require manual calculation of distances, leading to human errors and inefficiencies. Existing systems like Google Maps rely on complex APIs and real-time data, which may not be suitable for academic simulation.

2.Proposed System

The proposed system applies Dijkstra's Algorithm to a weighted graph representing cities. It provides accurate shortest paths with minimal computation time and serves as a simplified model of modern GPS systems.

Objectives:

1. To design a system that can represent cities and their connecting routes using graph data structures.
2. To implement Dijkstra's algorithm for computing the shortest path between two cities.
3. To compare routes based on different parameters such as distance or time.
4. To develop an interactive system that accepts user input for selecting source and destination cities.
5. To explore extensions like adding live traffic or dynamic weight updates.

3]SYSTEM DESIGN/METHODOLOGY:

The system design focuses on representing the map as a graph using either adjacency lists or matrices. Each city acts as a node, and every road connecting them is an edge with a specific weight representing distance or time. The Dijkstra algorithm is then applied to find the optimal path.

The head pointer points to the first student.

New students are added at the end of the list.

Traversal helps in displaying, searching, or sorting records.

Algorithm Used:

Dijkstra's Algorithm Steps:

1. Assign a tentative distance value to every vertex. Set it to zero for the starting point and infinity for all others.
2. Mark all vertices unvisited. Set the initial node as current.
3. For the current node, consider all unvisited neighbors and calculate tentative distances.
4. After visiting all neighbors, mark the current node as visited.
5. Select the unvisited node with the smallest tentative distance and repeat steps 3–4.
6. Continue until the destination node is marked visited.

Pseudocode

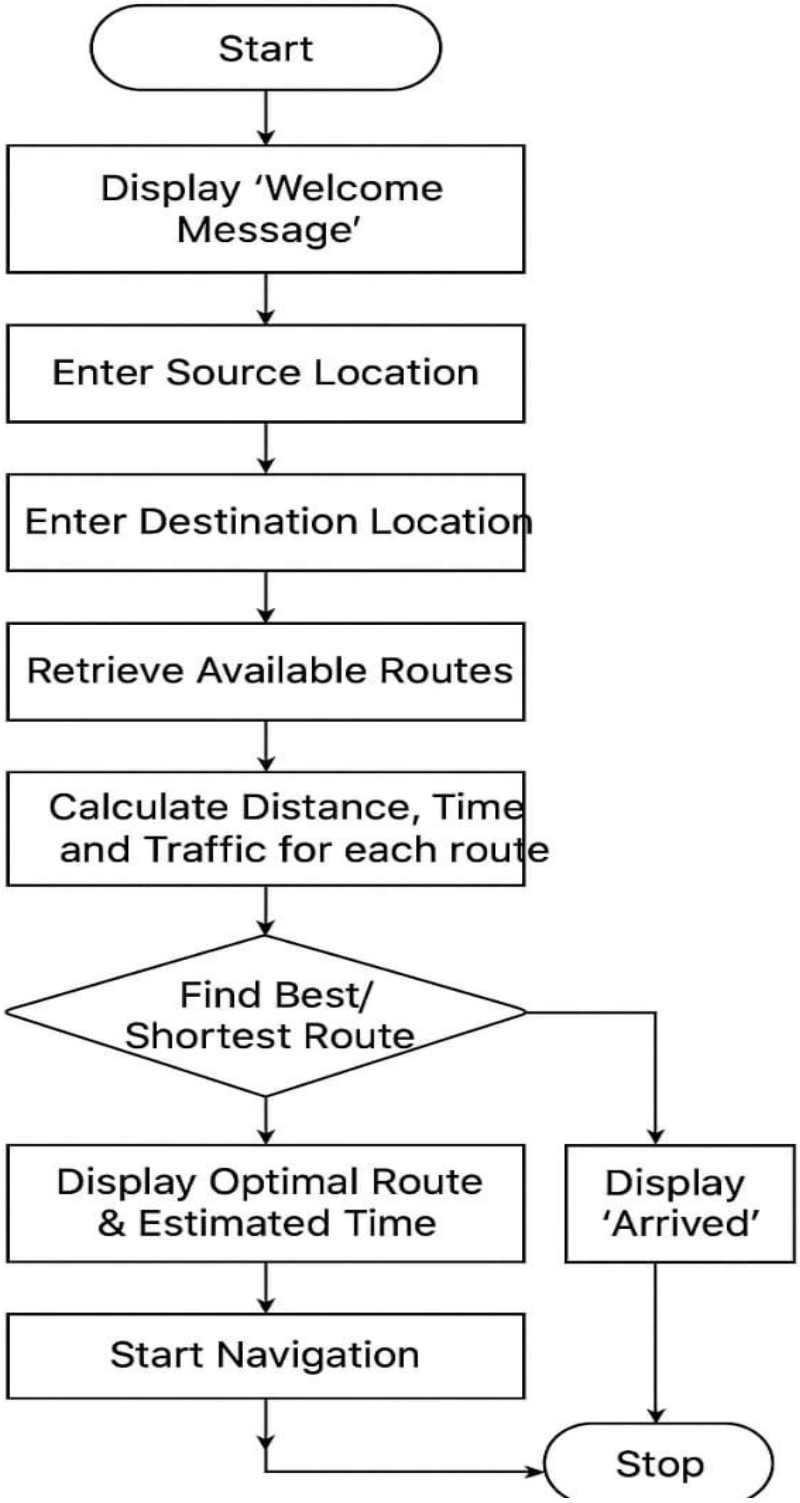
Procedure Dijkstra(Graph, source):

```
1. Initialize distances[] = ∞, distances[source] = 0
2. Set all nodes as unvisited
3. While unvisited nodes exist:
    a. Pick node u with smallest distance
    b. For each neighbor v of u:
        if distances[u] + weight(u,v) < distances[v]:
            distances[v] = distances[u] + weight(u,v)
   c. Mark u as visited
4. Return distances[]
```

Time Complexity

$O(V^2)$ for adjacency matrix; $O(E \log V)$ using adjacency list with priority queue.

FLOWCHART


Start
Display 'Welcome Message'
Enter Source Location
Enter Destination Location
Retrieve Available Routes
Calculate Distance, Time and Traffic for each route
Find Best/ Shortest Route
Display Optimal Route & Estimated Time
Display 'Arrived'
Start Navigation
Stop

4]IMPLEMENTATION DETAILS:

The implementation uses an adjacency matrix to represent the graph. Users can input the number of cities, their names, and the distances between them. After providing source and destination cities, Dijkstra's algorithm computes the shortest path and displays the result.

PROGRAM:

IMPLEMENTATION USING C(BACKEND LOGIC):

(FINDING SHORTEST PATH USING DIJKSTRAS ALGORITHM)

```
#include <stdio.h>

#include <limits.h>

#include <stdlib.h>


#define NUM_CITIES 5

const char* cityNames[NUM_CITIES] = {

   "Kailasagiri",

   "Simhachalam",

   "Bheemunipatnam",

   "Thagarapuvalasa",

   "Dwarakanagar"

};

  int findClosestCity(int weights[], int visited[]) {

   int minWeight = INT_MAX;

   int closest = -1;

   int i;

   for ( i = 0; i < NUM_CITIES; i++) {
```

```
        if (!visited[i] && weights[i] < minWeight) {
            minWeight = weights[i];
            closest = i;
        }
    }
    return closest;
}
void displayRoute(int parents[], int city) {
    if (parents[city] == -1) return;
    displayRoute(parents, parents[city]);
    printf(" -> %s", cityNames[city]);
}
void executeDijkstra(int graph[NUM_CITIES][NUM_CITIES], int start, int end,
char* type) {
    int dist[NUM_CITIES];
    int visited[NUM_CITIES];
    int parents[NUM_CITIES];
     int i;
    for ( i = 0; i < NUM_CITIES; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
        parents[i] = -1;
    }

    dist[start] = 0;
    int count;
```

```
    for ( count = 0; count < NUM_CITIES - 1; count++) {
        int u = findClosestCity(dist, visited);
        if (u == -1) break;
        visited[u] = 1;
        int v;
        for ( v = 0; v < NUM_CITIES; v++) {
            int weightThroughU = dist[u] + graph[u][v];
            if (!visited[v] && graph[u][v] > 0 && dist[u] != INT_MAX &&
weightThroughU < dist[v]) {
                dist[v] = weightThroughU;
                parents[v] = u;
            }
        }
    }

    if (dist[end] == INT_MAX) {
        printf("No available path from %s to %s.\n", cityNames[start],
cityNames[end]);
        return;
    }

    printf("Shortest %s from %s to %s: %d %s\n", type, cityNames[start],
cityNames[end], dist[end], type);
    printf("Route: %s", cityNames[start]);
    displayRoute(parents, end);
    printf("\n");
}
```

```
int main() {
    int distanceGraph[NUM_CITIES][NUM_CITIES] = {
        {0, 15, 10, 17, 9},
        {15, 0, 10, 0, 12},
        {10, 10, 0, 12, 5},
        {17, 0, 12, 0, 8},
        {9, 12, 5, 8, 0}
    };

    int timeGraph[NUM_CITIES][NUM_CITIES] = {
        {0, 20, 12, 25, 15},
        {20, 0, 12, 0, 18},
        {12, 12, 0, 15, 8},
        {25, 0, 15, 0, 10},
        {15, 18, 8, 10, 0}
    };

    printf("City List:\n");
    int i;
    for ( i = 0; i < NUM_CITIES; i++)
        printf("%d - %s\n", i, cityNames[i]);
        int choice;
    printf("Choose calculation: 1 - Distance, 2 - Time: ");
    scanf("%d", &choice);
```

```
    int source, destination;
    printf("Enter source city number: ");
    scanf("%d", &source);
    printf("Enter destination city number: ");
    scanf("%d", &destination);

    if (source == destination) {
        printf("You are already at %s!\n", cityNames[source]);
        return 0;
    }
        if (choice == 1)
        executeDijkstra(distanceGraph, source, destination, "Distance");
    else if (choice == 2)
        executeDijkstra(timeGraph, source, destination, "Time");
    else
        printf("Invalid choice.\n");

    return 0;
}
```

EXPLANATION OF CODE SNIPETS:

1. Header Files and Macros

```
#include <stdio.h>
#include <limits.h>
```

```
#include <stdlib.h>

#define NUM_CITIES
```

- `stdio.h: For input/output functions (printf, scanf).`
- `limits.h: For constants like INT_MAX (represents infinity).`
- `stdlib.h: For general utilities`
- `NUM_CITIES 5: There are 5 cities total.`

2. City Names Array

```
const char* cityNames[NUM_CITIES] = {

    "Kailasagiri",

    "Simhachalam",

    "Bheemunipatnam",

    "Thagarapuvalasa",

    "Dwarakanagar"

};
```

- Stores city names as strings for display purposes.

3. findClosestCity() Function

```
int findClosestCity(int weights[], int visited[]) {

int minWeight = INT_MAX;

int closest = -1;

int i;

for (i = 0; i < NUM_CITIES; i++) {

    if (!visited[i] && weights[i] < minWeight) {

        minWeight = weights[i];
```

```
            closest = i;
        }
    }
    return closest;
}
```

Purpose: Finds the unvisited city that currently has the smallest tentative distance/time.

How it works:

- It loops through all cities.
- Skips already visited ones.
- Finds the city with the smallest weight (distance/time) value.
- Returns that city's index.

This is a key part of Dijkstra's algorithm.

4. displayRoute() Function

```
void displayRoute(int parents[], int city) {
if (parents[city] == -1) return;
displayRoute(parents, parents[city]);
printf(" -> %s", cityNames[city]);
}
```

Purpose: Prints the full route from the start city to the end city.

How it works:

- It uses recursion.
- Each city keeps track of its parent city (the previous step in the shortest path).
- The function traces backward until it reaches the starting city, then prints the route in correct order.

5. executeDijkstra() Function

This is the main algorithm.

```
void executeDijkstra(int graph[NUM_CITIES][NUM_CITIES], int start, int end,
char* type) {

    int dist[NUM_CITIES];

    int visited[NUM_CITIES];

    int parents[NUM_CITIES];

    int i;

}
```

- graph: The matrix of distances or times.
- dist[]: Holds the shortest distance/time from the start to each city.
- visited[]: Marks whether each city is finalized.
- parents[]: Stores the previous city for reconstructing the path.

Initialization

```
for (i = 0; i < NUM_CITIES; i++) {

    dist[i] = INT_MAX;

    visited[i] = 0;

    parents[i] = -1;

}

dist[start] = 0;
```

All distances are set to infinity (INT_MAX), except for the starting city which is 0.

Main Loop

```
for (count = 0; count < NUM_CITIES - 1; count++) {

    int u = findClosestCity(dist, visited);

    if (u == -1) break;

    visited[u] = 1;



    for (v = 0; v < NUM_CITIES; v++) {

        int weightThroughU = dist[u] + graph[u][v];

        if (!visited[v] && graph[u][v] > 0 && dist[u] != INT_MAX &&
weightThroughU < dist[v]) {

            dist[v] = weightThroughU;

            parents[v] = u;

        }

    }

}
```

This is Dijkstra's algorithm:

1. Pick the nearest unvisited city u.
2. Mark it as visited.
3. Update its neighbors' distances if a shorter path is found via u.
4. Repeat until all cities are processed or destination is reached.

Display Result

```
if (dist[end] == INT_MAX) {

    printf("No available path from %s to %s.\n", cityNames[start], cityNames[end]);

    return;

}
```

```
printf("Shortest %s from %s to %s: %d %s\n", type, cityNames[start], cityNames[end], dist[end], type);

printf("Route: %s", cityNames[start]);

displayRoute(parents, end);

printf("\n");
```

- Prints the shortest distance or time.
- Shows the complete route (by recursively printing the path).

6. main() Function

```
int main() {

    int distanceGraph[NUM_CITIES][NUM_CITIES] = {...};

    int timeGraph[NUM_CITIES][NUM_CITIES] = {...};
```

Two matrices are created:

- distanceGraph → represents distance (km)
- timeGraph → represents time (minutes)

Displaying the City List

```
printf("City List:\n");

for (i = 0; i < NUM_CITIES; i++)

    printf("%d - %s\n", i, cityNames[i]);
```

Lists cities with their index numbers for the user to select.

User Input

```
int choice;

printf("Choose calculation: 1 - Distance, 2 - Time: ");
```

```
scanf("%d", &choice);


int source, destination;

printf("Enter source city number: ");

scanf("%d", &source);

printf("Enter destination city number: ");

scanf("%d", &destination);
```

Takes user input for:

- Type of calculation (distance or time)
- Source city
- Destination city

Validation and Execution

```
if (source == destination) {

    printf("You are already at %s!\n", cityNames[source]);

    return 0;

}


if (choice == 1)

    executeDijkstra(distanceGraph, source, destination, "Distance");

else if (choice == 2)

    executeDijkstra(timeGraph, source, destination, "Time");

else

    printf("Invalid choice.\n");
```

- If both cities are same → message displayed.
- Otherwise, runs Dijkstra with the chosen graph.

IMPLEMENTATION USING PYTHON(FRONT END VISUALIZATION):

(GRAPH VISUALIZATION USING TKINTER)

```
import tkinter as tk
from tkinter import ttk, messagebox

cityNames = ["Kailasagiri", "Simhachalam", "Bheemunipatnam",
"Thagarapuvalasa", "Dwarakanagar"]
NUM_CITIES = len(cityNames)

distanceGraph = [
    [0, 15, 30, 37, 15],
    [15, 0, 31, 35, 25],
    [30, 31, 0, 11, 30],
    [37, 35, 11, 0, 30],
    [15, 25, 30, 30, 0]
]

timeGraph = [
    [0, 25, 50, 60, 25],
    [25, 0, 60, 55, 40],
    [50, 60, 0, 20, 45],
    [60, 55, 20, 0, 45],
    [25, 40, 45, 45, 0]
]

def dijkstra(graph, start, end):
    distances = [float('inf')] * NUM_CITIES
    visited = [False] * NUM_CITIES
    parents = [-1] * NUM_CITIES
    distances[start] = 0

    for _ in range(NUM_CITIES):
        min_dist = float('inf')
        u = -1
```

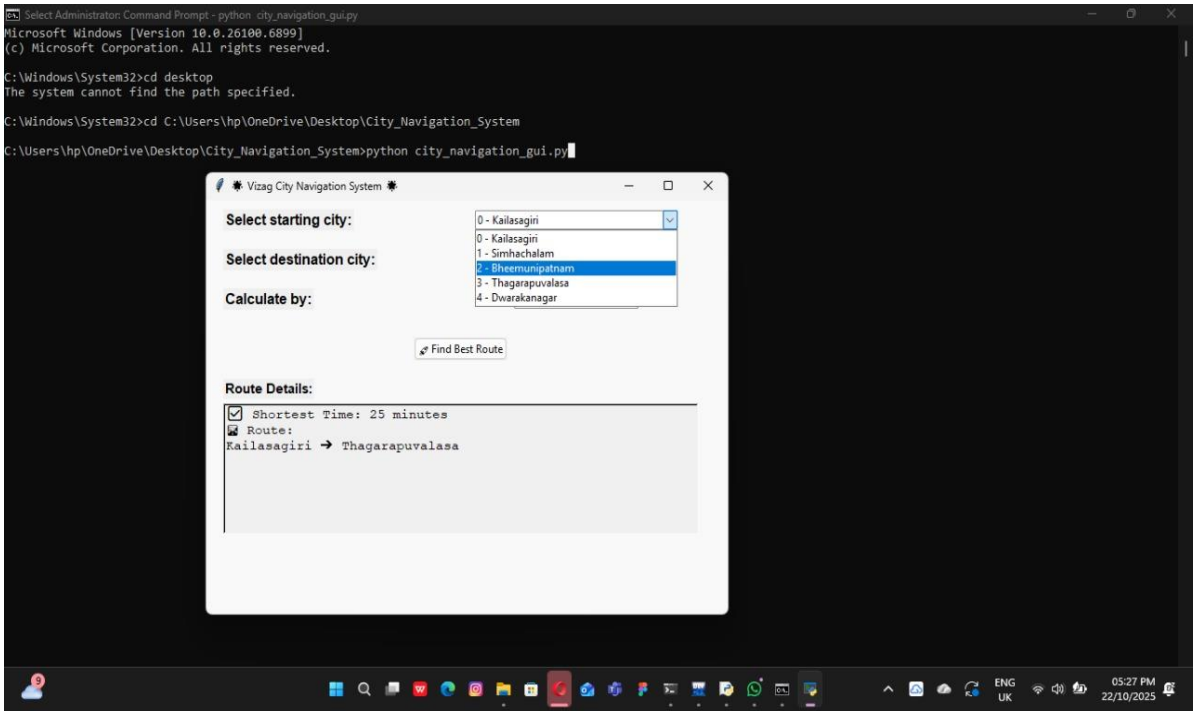
```
        for i in range(NUM_CITIES):
            if not visited[i] and distances[i] < min_dist:
                min_dist = distances[i]
                u = i
        if u == -1:
            break
        visited[u] = True
        for v in range(NUM_CITIES):
            if graph[u][v] > 0 and not visited[v]:
                if distances[u] + graph[u][v] < distances[v]:
                    distances[v] = distances[u] + graph[u][v]
                    parents[v] = u

    path = []
    current = end
    if distances[end] == float('inf'):
        return None, None
    while current != -1:
        path.insert(0, cityNames[current])
        current = parents[current]
    return distances[end], path

def calculate_route():
    start_idx = source_var.get()
    end_idx = dest_var.get()
    calc_type = calc_var.get()
    if start_idx == end_idx:
        messagebox.showinfo("Ooops!", f"You are already at
{cityNames[start_idx]}!")
        return
    graph = distanceGraph if calc_type == "Distance" else timeGraph
    distance, path = dijkstra(graph, start_idx, end_idx)
    if distance is None:
        result_label.config(text=f"there is no available route from
{cityNames[start_idx]} to {cityNames[end_idx]} ")
    else:
        route_str = " ➔ ".join(path)
        unit = "km" if calc_type == "Distance" else "minutes"
        result_label.config(text=f" Shortest {calc_type}: {distance} {unit}\n
route:\n{route_str}")
```

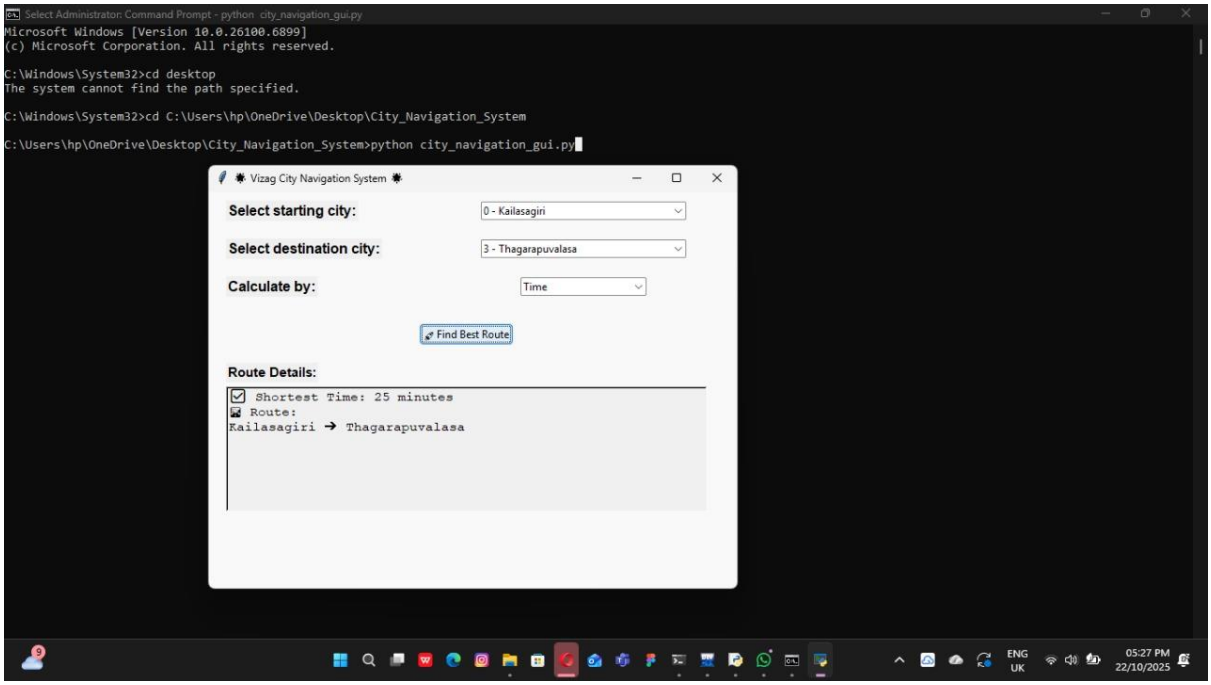
```
root = tk.Tk()
root.title("-- Vizag City Navigation System --")
root.geometry("600x450")
root.configure(bg="#f9f9f9")

ttk.Label(root, text="Select starting city:", font=("Arial", 12,
"bold")).grid(row=0, column=0, padx=20, pady=10, sticky="w")
source_var = tk.IntVar()
source_dropdown = ttk.Combobox(root, values=[f"{i} - {c}" for i,c in
enumerate(cityNames)], state="readonly", width=35)
source_dropdown.grid(row=0, column=1, padx=20, pady=10)
source_dropdown.current(0)
source_var.set(0)
source_dropdown.bind("<<ComboboxSelected>>", lambda e:
source_var.set(source_dropdown.current()))

ttk.Label(root, text="Select destination city:", font=("Arial", 12,
"bold")).grid(row=1, column=0, padx=20, pady=10, sticky="w")
dest_var = tk.IntVar()
dest_dropdown = ttk.Combobox(root, values=[f"{i} - {c}" for i,c in
enumerate(cityNames)], state="readonly", width=35)
dest_dropdown.grid(row=1, column=1, padx=20, pady=10)
dest_dropdown.current(1)
dest_var.set(1)
dest_dropdown.bind("<<ComboboxSelected>>", lambda e:
dest_var.set(dest_dropdown.current()))

# Calculation type
ttk.Label(root, text="Calculate by:", font=("Arial", 12, "bold")).grid(row=2,
column=0, padx=20, pady=10, sticky="w")
calc_var = tk.StringVar()
calc_dropdown = ttk.Combobox(root, values=["Distance", "Time"],
state="readonly", width=20)
calc_dropdown.grid(row=2, column=1, padx=20, pady=10)
calc_dropdown.current(0)
calc_var.set("Distance")
calc_dropdown.bind("<<ComboboxSelected>>", lambda e:
calc_var.set(calc_dropdown.get()))
```

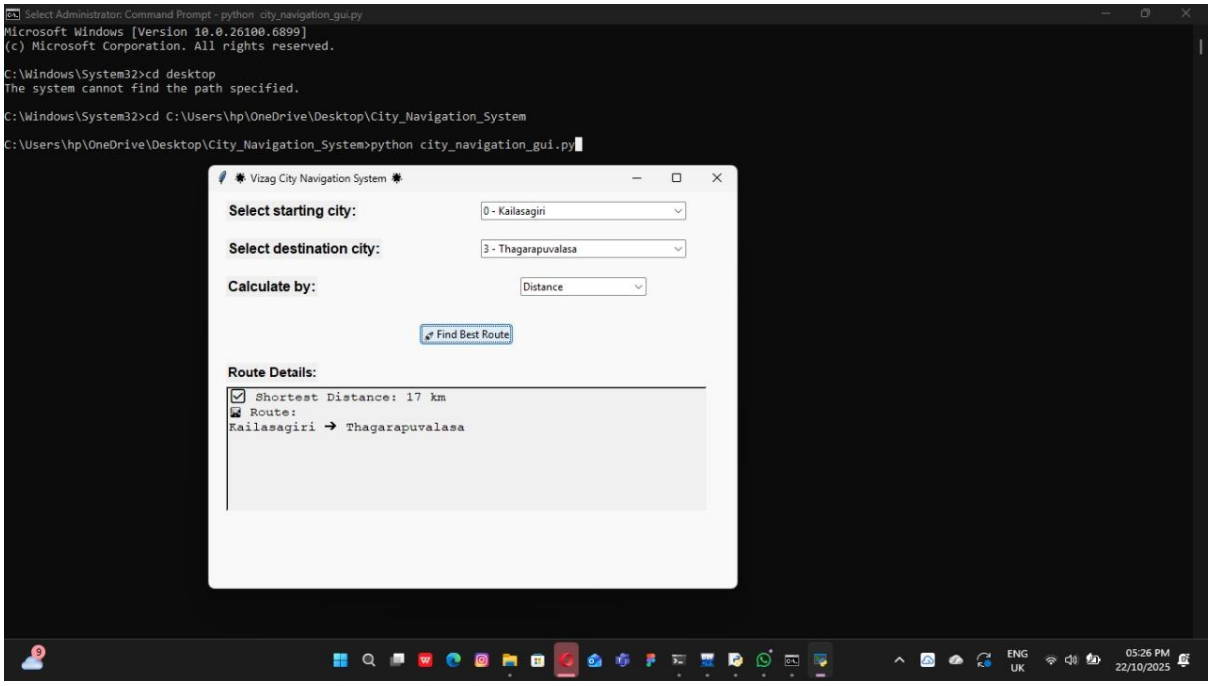
```
# Button
ttk.Button(root, text=" Find Best Route",
command=calculate_route).grid(row=3, column=0, columnspan=2, pady=20)

# Result display
ttk.Label(root, text="Routingg Details:", font=("Arial", 11, "bold")).grid(row=4,
column=0, sticky="w", padx=20)
result_label = tk.Label(root, text="", font=("Courier", 11), justify="left",
bg="#f0f0f0", anchor="nw", width=60, height=8, bd=2, relief="sunken")
result_label.grid(row=5, column=0, columnspan=2, padx=20, pady=5,
sticky="w")

root.mainloop()
```

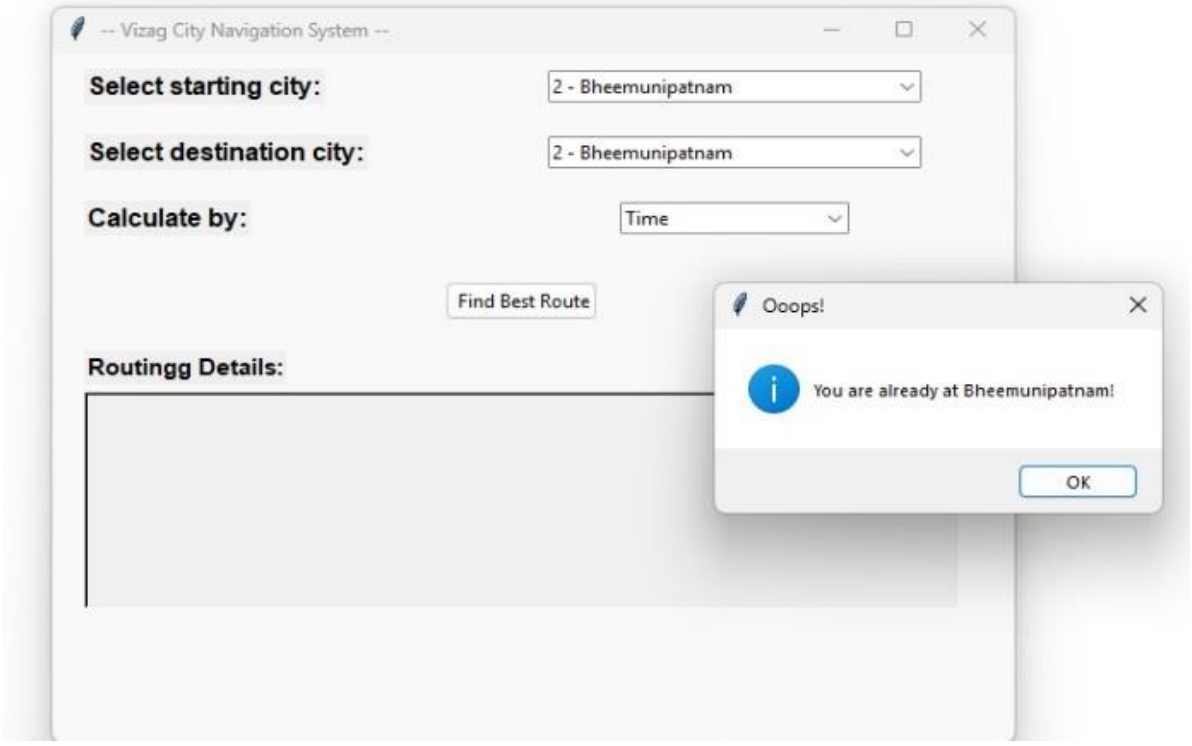
5]RESULTS AND DISCUSSIONS

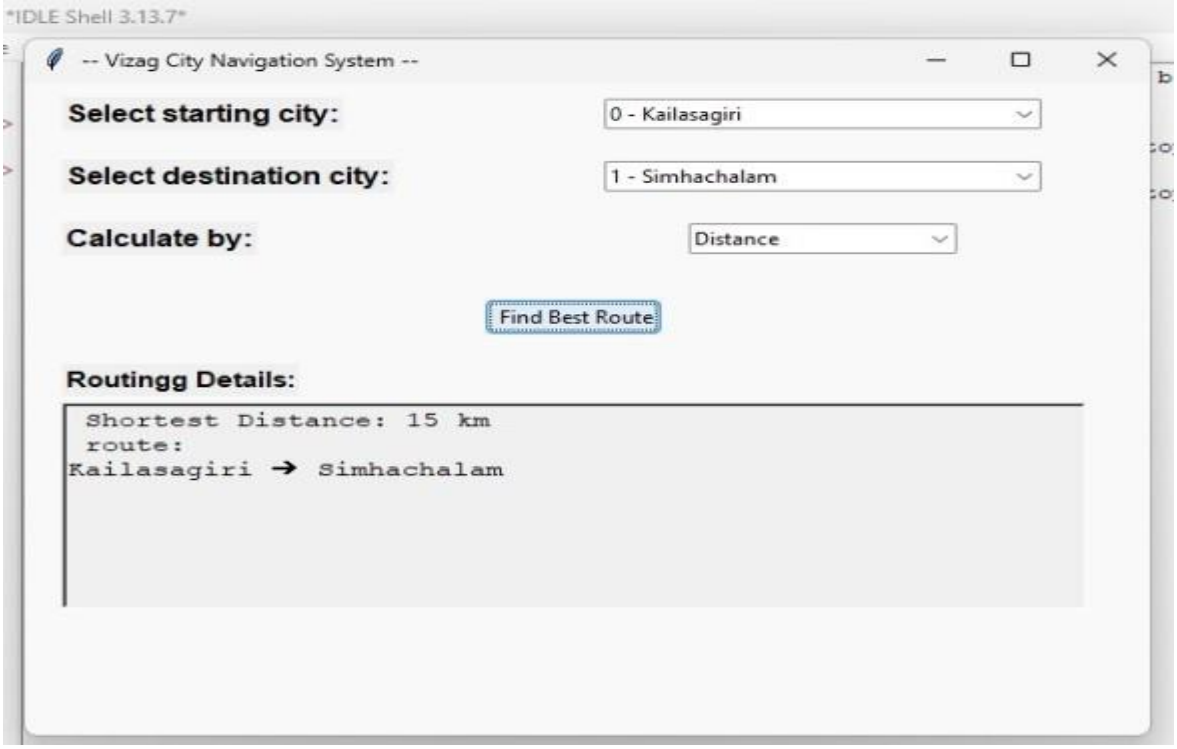
Output:

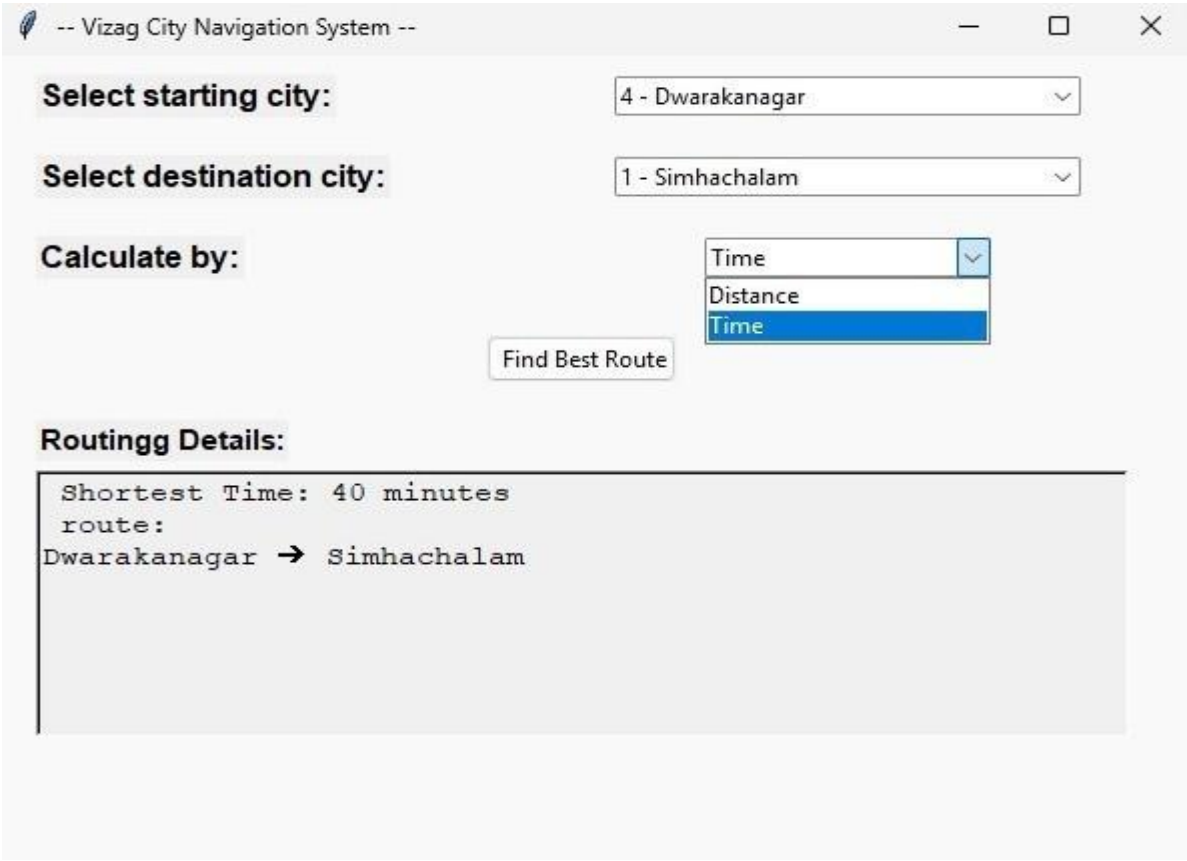





Select Administrator: Command Prompt - python city_navigation_gui.py
Microsoft Windows [Version 10.0.26100.6899]
(c) Microsoft Corporation. All rights reserved.
C:\Windows\System32>cd desktop
The system cannot find the path specified.
C:\Windows\System32>cd C:\Users\hp\OneDrive\Desktop\City_Navigation_System
C:\Users\hp\OneDrive\Desktop\City_Navigation_System>python city_navigation_gui.py
Vizag City Navigation System
Select starting city:
0 - Kailasagiri
Select destination city:
3 - Thagarapuvalasa
Calculate by:
Distance
Find Best Route
Route Details:
Shortest Distance: 17 km
Route:
Kailasagiri → Thagarapuvalasa


-- Vizag City Navigation System --
Select starting city:
2 - Bheemunipatnam
Select destination city:
2 - Bheemunipatnam
Calculate by:
Time
Find Best Route
Routingg Details:
Ooops!
You are already at Bheemunipatnam!
OK


IDLE Shell 3.13.7
-- Vizag City Navigation System --
Select starting city:
0 - Kailasagiri
Select destination city:
1 - Simhachalam
Calculate by:
Distance
Find Best Route
Routingg Details:
Shortest Distance: 15 km
route:
Kailasagiri → Simhachalam


-- Vizag City Navigation System --
Select starting city:
4 - Dwarakanagar
Select destination city:
1 - Simhachalam
Calculate by:
Time
Distance
Time
Find Best Route
Routingg Details:
Shortest Time: 40 minutes
route:
Dwarakanagar → Simhachalam

PLAGARISM

C CODE

Sentence wise results | Matched Sources

```
#include
#define Cities 10
#define INFINITY_VAL 1e9f
#define RATE_PER_KM 12.0f
// Tourist places considered as "spots"
const char *spots[Cities] = {
  "RK Beach",
  "Submarine Museum",
  "Kailasagiri Hill Park",
  "Rishikonda Beach",
  "Yarada Beach",
  "Dolphin's Nose",
```

Save Report | Download Report

96% Unique Content

4% Plagiarized content

✔ COMPLETED

100%

PYTHON CODE:

```
import tkinter as tk
from tkinter import ttk, messagebox
cityNames = ["Kailasagiri", "Simhachalam", "Bheemunipatnam", "Thagarapuvalasa",
"Dwarakanagar"]
NUM_CITIES = len(cityNames)
distanceGraph = [
  [0, 15, 30, 37, 15],
  [15, 0, 31, 35, 25],
  [30, 31, 0, 11, 30],
  [37, 35, 11, 0, 30],
```

100% Unique Content

0% Plagiarized content

✔ COMPLETED

100%

6] Conclusion :

The City Navigation System demonstrates how Dijkstra's Algorithm can determine the shortest path between cities, providing a simplified model of navigation systems used in real life. The project also enhances understanding of graph-based algorithms.

FUTURE ENHANCEMENTS

1. Integration with Real-Time Traffic Data

2. Graphical User Interface (GUI)

3. Multiple Optimization Criteria

4. Dynamic Graph Updates

5. Multi-Destination and Multi-User Routing

APPLICATIONS

The **City Navigation System** has a wide range of real-world and educational applications.
By applying graph theory and Dijkstra's algorithm, the project demonstrates how computational models solve real-life navigation problems efficiently.
1. Real-World Navigation
2. Logistics and Delivery Optimization
3. Traffic and Transport Management
4. Network Routing in Communication Systems
5. Educational Use

LIMITATIONS

1. Static Data:
2. Limited Scalability:

7]REFERENCES:

1. GeeksforGeeks – Dijkstra's Algorithm

2. TutorialsPoint – Graph Data Structures

3. Data Structures Textbook by Reema Thareja

4. C Programming Language by E. Balagurusamy

8]APPENDIX:

C code

```
#include <stdio.h>

#include <limits.h>

#include <stdlib.h>


#define NUM_CITIES 5

const char* cityNames[NUM_CITIES] = {

    "Kailasagiri",

    "Simhachalam",

    "Bheemunipatnam",

    "Thagarapuvalasa",

    "Dwarakanagar"

};

   int findClosestCity(int weights[], int visited[]) {

    int minWeight = INT_MAX;

    int closest = -1;

    int i;

    for ( i = 0; i < NUM_CITIES; i++) {

        if (!visited[i] && weights[i] < minWeight) {

            minWeight = weights[i];

            closest = i;

        }

    }

    return closest;
```

```
}
void displayRoute(int parents[], int city) {
    if (parents[city] == -1) return;
    displayRoute(parents, parents[city]);
    printf(" -> %s", cityNames[city]);
}
void executeDijkstra(int graph[NUM_CITIES][NUM_CITIES], int start, int end,
char* type) {
    int dist[NUM_CITIES];
    int visited[NUM_CITIES];
    int parents[NUM_CITIES];
     int i;
    for ( i = 0; i < NUM_CITIES; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
        parents[i] = -1;
    }

    dist[start] = 0;
    int count;

    for ( count = 0; count < NUM_CITIES - 1; count++) {
        int u = findClosestCity(dist, visited);
        if (u == -1) break;
        visited[u] = 1;
        int v;
        for ( v = 0; v < NUM_CITIES; v++) {
```

```
            int weightThroughU = dist[u] + graph[u][v];
            if (!visited[v] && graph[u][v] > 0 && dist[u] != INT_MAX &&
weightThroughU < dist[v]) {
                dist[v] = weightThroughU;
                parents[v] = u;
            }
        }
    }

    if (dist[end] == INT_MAX) {
        printf("No available path from %s to %s.\n", cityNames[start],
cityNames[end]);
        return;
    }

    printf("Shortest %s from %s to %s: %d %s\n", type, cityNames[start],
cityNames[end], dist[end], type);
    printf("Route: %s", cityNames[start]);
    displayRoute(parents, end);
    printf("\n");
}


int main() {
    int distanceGraph[NUM_CITIES][NUM_CITIES] = {
        {0, 15, 10, 17, 9},
        {15, 0, 10, 0, 12},
```

```
    {10, 10, 0, 12, 5},
    {17, 0, 12, 0, 8},
    {9, 12, 5, 8, 0}
};

int timeGraph[NUM_CITIES][NUM_CITIES] = {
    {0, 20, 12, 25, 15},
    {20, 0, 12, 0, 18},
    {12, 12, 0, 15, 8},
    {25, 0, 15, 0, 10},
    {15, 18, 8, 10, 0}
};

printf("City List:\n");
int i;
for ( i = 0; i < NUM_CITIES; i++)
    printf("%d - %s\n", i, cityNames[i]);
    int choice;
printf("Choose calculation: 1 - Distance, 2 - Time: ");
scanf("%d", &choice);


int source, destination;
printf("Enter source city number: ");
scanf("%d", &source);
printf("Enter destination city number: ");
scanf("%d", &destination);
```

```
    if (source == destination) {
        printf("You are already at %s!\n", cityNames[source]);
        return 0;
    }
        if (choice == 1)
        executeDijkstra(distanceGraph, source, destination, "Distance");
    else if (choice == 2)
        executeDijkstra(timeGraph, source, destination, "Time");
    else
        printf("Invalid choice.\n");


    return 0;
}
```

Python code

```
import tkinter as tk
from tkinter import ttk, messagebox

cityNames = ["Kailasagiri", "Simhachalam", "Bheemunipatnam",
"Thagarapuvalasa", "Dwarakanagar"]
NUM_CITIES = len(cityNames)

distanceGraph = [
    [0, 15, 30, 37, 15],
    [15, 0, 31, 35, 25],
    [30, 31, 0, 11, 30],
    [37, 35, 11, 0, 30],
    [15, 25, 30, 30, 0]
]
```

```
timeGraph = [
    [0, 25, 50, 60, 25],
    [25, 0, 60, 55, 40],
    [50, 60, 0, 20, 45],
    [60, 55, 20, 0, 45],
    [25, 40, 45, 45, 0]
]

def dijkstra(graph, start, end):
    distances = [float('inf')] * NUM_CITIES
    visited = [False] * NUM_CITIES
    parents = [-1] * NUM_CITIES
    distances[start] = 0

    for _ in range(NUM_CITIES):
        min_dist = float('inf')
        u = -1
        for i in range(NUM_CITIES):
            if not visited[i] and distances[i] < min_dist:
                min_dist = distances[i]
                u = i
        if u == -1:
            break
        visited[u] = True
        for v in range(NUM_CITIES):
            if graph[u][v] > 0 and not visited[v]:
                if distances[u] + graph[u][v] < distances[v]:
                    distances[v] = distances[u] + graph[u][v]
                    parents[v] = u

    path = []
    current = end
    if distances[end] == float('inf'):
        return None, None
    while current != -1:
        path.insert(0, cityNames[current])
        current = parents[current]
    return distances[end], path

def calculate_route():
```

```
    start_idx = source_var.get()
    end_idx = dest_var.get()
    calc_type = calc_var.get()
    if start_idx == end_idx:
        messagebox.showinfo("Ooops!", f"You are already at
{cityNames[start_idx]}!")
        return
    graph = distanceGraph if calc_type == "Distance" else timeGraph
    distance, path = dijkstra(graph, start_idx, end_idx)
    if distance is None:
        result_label.config(text=f"there is no available route from
{cityNames[start_idx]} to {cityNames[end_idx]} ")
    else:
        route_str = " ➔ ".join(path)
        unit = "km" if calc_type == "Distance" else "minutes"
        result_label.config(text=f" Shortest {calc_type}: {distance} {unit}\n
route:\n{route_str}")

root = tk.Tk()
root.title("-- Vizag City Navigation System --")
root.geometry("600x450")
root.configure(bg="#f9f9f9")

ttk.Label(root, text="Select starting city:", font=("Arial", 12,
"bold")).grid(row=0, column=0, padx=20, pady=10, sticky="w")
source_var = tk.IntVar()
source_dropdown = ttk.Combobox(root, values=[f"{i} - {c}" for i,c in
enumerate(cityNames)], state="readonly", width=35)
source_dropdown.grid(row=0, column=1, padx=20, pady=10)
source_dropdown.current(0)
source_var.set(0)
source_dropdown.bind("<<ComboboxSelected>>", lambda e:
source_var.set(source_dropdown.current()))

ttk.Label(root, text="Select destination city:", font=("Arial", 12,
"bold")).grid(row=1, column=0, padx=20, pady=10, sticky="w")
dest_var = tk.IntVar()
dest_dropdown = ttk.Combobox(root, values=[f"{i} - {c}" for i,c in
enumerate(cityNames)], state="readonly", width=35)
dest_dropdown.grid(row=1, column=1, padx=20, pady=10)
```

```
dest_dropdown.current(1)
dest_var.set(1)
dest_dropdown.bind("<<ComboboxSelected>>", lambda e:
dest_var.set(dest_dropdown.current()))

# Calculation type
ttk.Label(root, text="Calculate by:", font=("Arial", 12, "bold")).grid(row=2,
column=0, padx=20, pady=10, sticky="w")
calc_var = tk.StringVar()
calc_dropdown = ttk.Combobox(root, values=["Distance", "Time"],
state="readonly", width=20)
calc_dropdown.grid(row=2, column=1, padx=20, pady=10)
calc_dropdown.current(0)
calc_var.set("Distance")
calc_dropdown.bind("<<ComboboxSelected>>", lambda e:
calc_var.set(calc_dropdown.get()))

# Button
ttk.Button(root, text=" Find Best Route",
command=calculate_route).grid(row=3, column=0, columnspan=2, pady=20)

# Result display
ttk.Label(root, text="Routingg Details:", font=("Arial", 11, "bold")).grid(row=4,
column=0, sticky="w", padx=20)
result_label = tk.Label(root, text="", font=("Courier", 11), justify="left",
bg="#f0f0f0", anchor="nw", width=60, height=8, bd=2, relief="sunken")
result_label.grid(row=5, column=0, columnspan=2, padx=20, pady=5,
sticky="w")

root.mainloop()
```